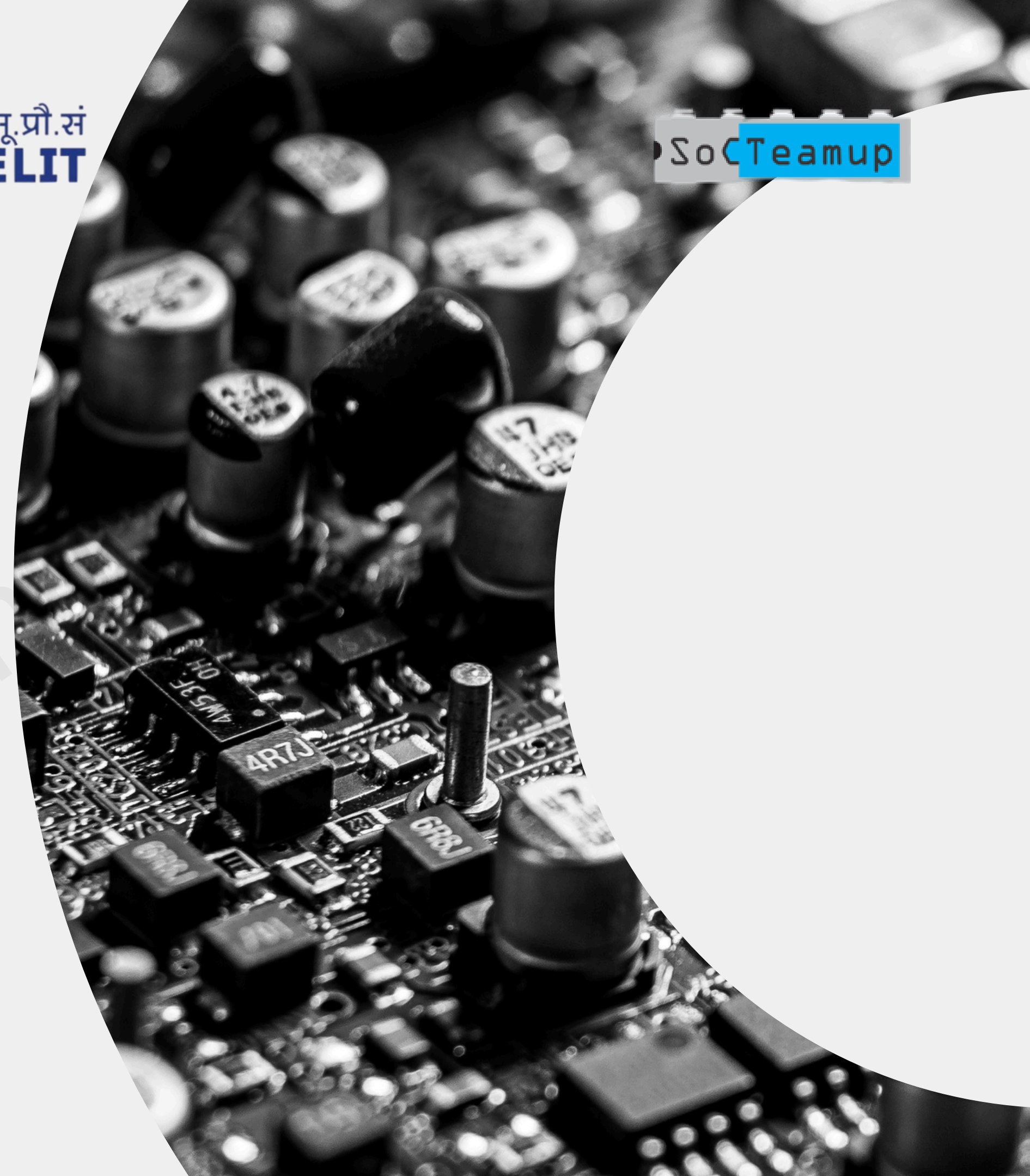


# Clock Tree Synthesis (CTS) concepts H-tree, buffered trees







# Outline

|                                              |
|----------------------------------------------|
| Introduction to Clock Tree Synthesis (CTS)   |
| Need for a Clock Distribution Network        |
| CTS Objectives and Challenges                |
| Clock Skew, Latency, and Jitter              |
| Clock Tree Topologies (Overview)             |
| H-tree Based Clock Distribution              |
| Buffered (Tree-Based) Clock Networks         |
| Basic CTS Steps in Open-Source Flow          |
| Sample TCL Commands for Clock Tree Insertion |
| Clock Tree Quality Metrics                   |
| Common Issues in CTS                         |
| CTS Optimization Techniques                  |
| Example Flow Summary                         |
| Summary and Best Practices                   |

# ➔ Introduction: What is Clock Tree Synthesis?

- **Clock Tree Synthesis (CTS)** is the process of distributing the clock signal from a single source (e.g., PLL or clock pin) to all sequential elements (flip-flops, registers) in the design.
- The goal is to ensure that all flip-flops receive the clock simultaneously, minimizing timing differences.
- CTS happens after placement and before routing.

**Why CTS  
matters:**

**Controls  
setup/hold  
timing**

**Impacts  
performance  
and power**

**Affects  
synchronizatio  
n reliability**

## Need for a Clock Distribution Network

- Without CTS, the clock would travel unevenly across the chip due to varying wire lengths and parasitics.

|                             |                                                          |
|-----------------------------|----------------------------------------------------------|
| Problems<br>Without<br>CTS: | Skew between clock arrival times at different flip-flops |
|                             | Increased timing violations                              |
|                             | Potential data corruption                                |

### **Solution:**

Introduce balanced, buffered clock networks to equalize delays.



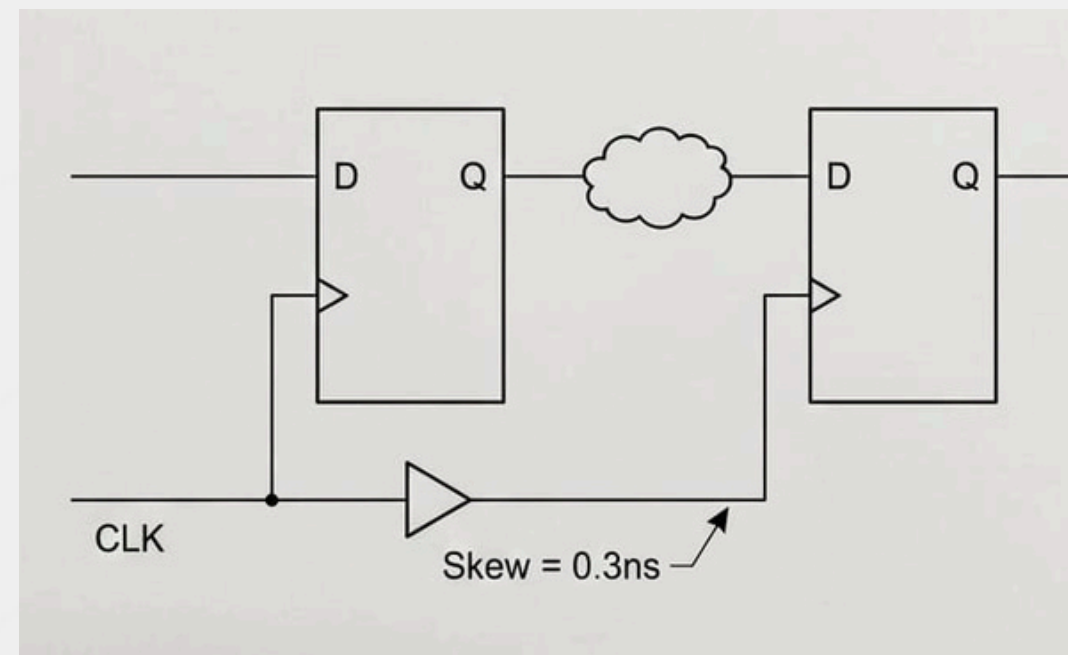
# → CTS Objectives

| Objective               | Description                                                          |
|-------------------------|----------------------------------------------------------------------|
| <b>Minimize Skew</b>    | Ensure minimal difference between earliest and latest clock arrival. |
| <b>Minimize Latency</b> | Reduce total delay from clock source to destination.                 |
| <b>Optimize Power</b>   | Minimize dynamic power due to switching.                             |
| <b>Balance Load</b>     | Keep buffer fanout uniform.                                          |
| <b>Avoid Crosstalk</b>  | Use shielded routing for key clock nets.                             |

**Tools aim to trade off** between skew, latency, and power to achieve optimal CTS.

## ➔ Clock Skew Example

Clock Source  
|---> FF1 (Arrival: 2ns)  
|---> FF2 (Arrival:  
2.3ns)



- **Clock Skew** = 0.3 ns
- **Positive Skew:** Clock arrives later at destination FF → helps setup time.
- **Negative Skew:** Clock arrives earlier → causes hold time violations.
- **CTS tries to minimize skew** by balancing wire lengths and buffer delays.



## Clock Tree Topologies

| Topology      | Description                                    |
|---------------|------------------------------------------------|
| H-Tree        | Symmetrical tree distributing clock uniformly. |
| Balanced Tree | Hierarchical structure with buffers.           |
| Spine Tree    | Central spine with branching lines.            |
| Grid Network  | Mesh-like structure for large chips.           |

**Choice depends on:** Chip size, number of sinks (flip-flops), and performance target.

# H-Tree Clock Network

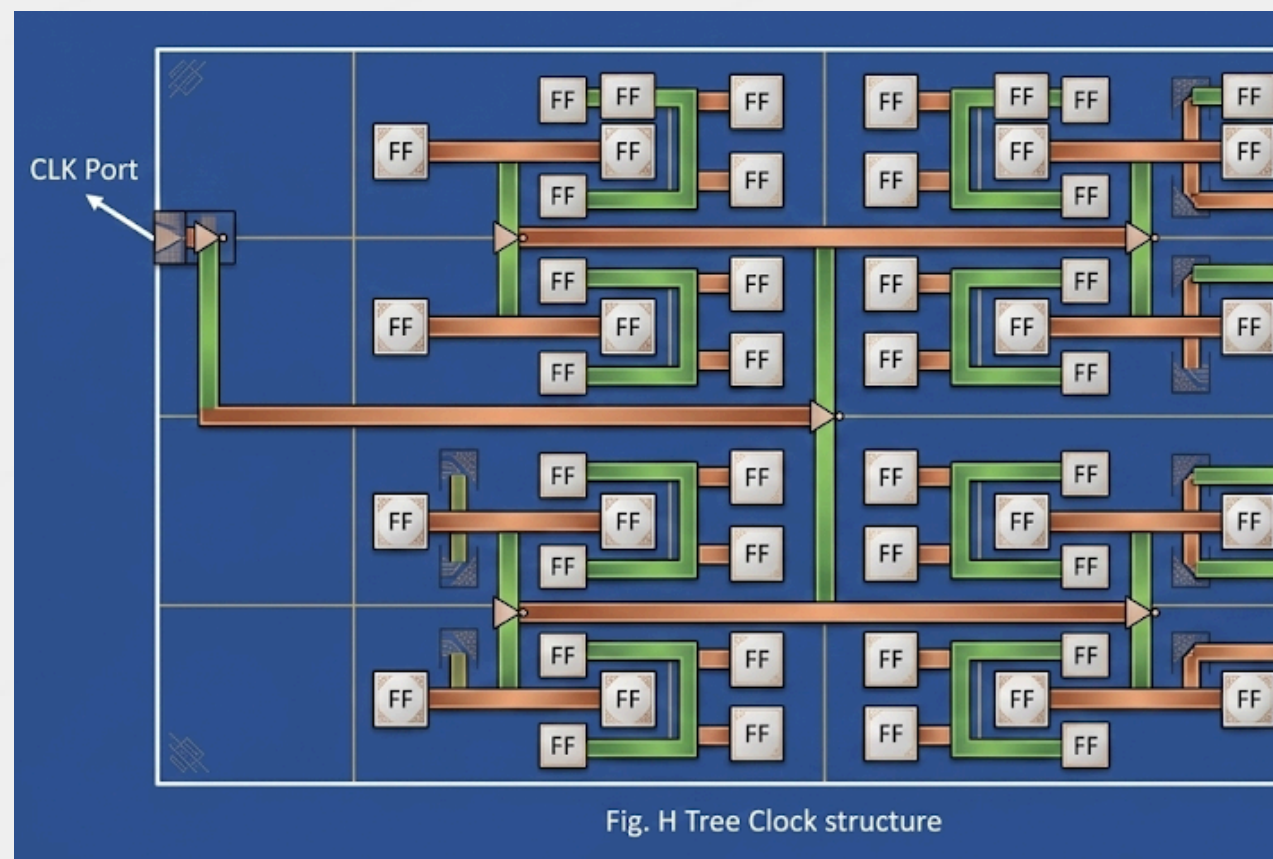
- H-tree is a geometrically balanced tree ensuring equal path length from the clock source to each sink.
- Common in regular layouts like arrays or processors.

## Features:

- Zero skew (theoretically)
- Symmetric branching
- Easy to implement with recursive division

# ➔ H-Tree Clock Network

- Concept Diagram:



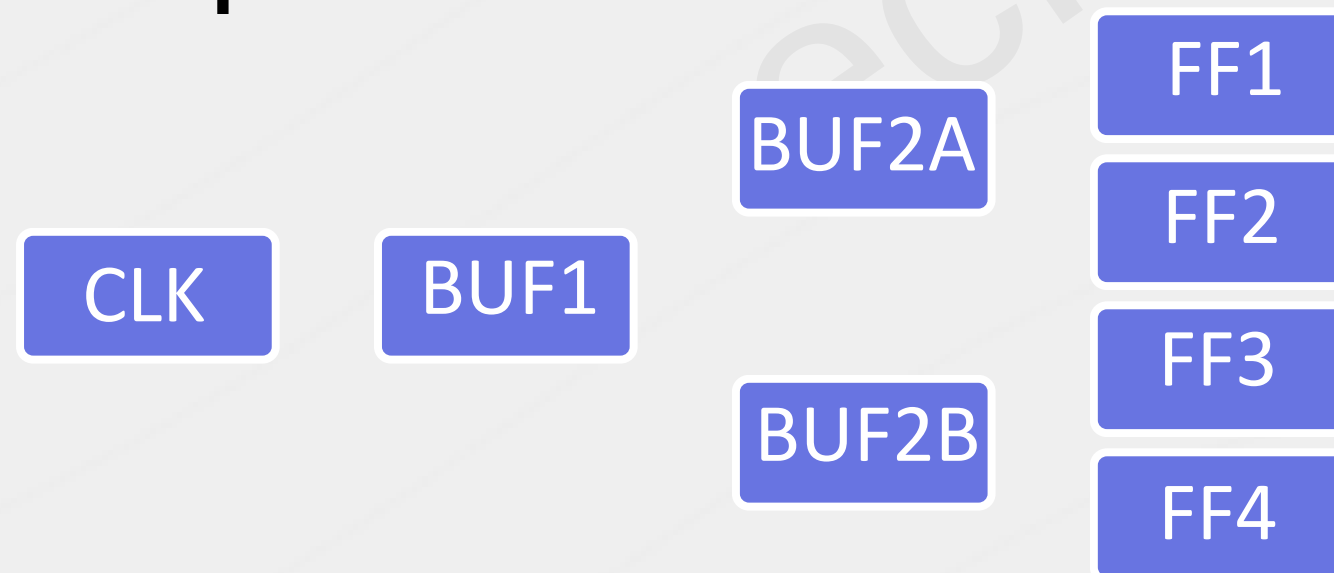
Each path from CLK → FF has equal length → minimal skew.



# ➔ Buffered Clock Tree

- Buffered trees insert buffers/inverters along clock paths to:
  - Drive large fanouts
  - Control delay
  - Reduce skew

## Example Concept:



## **Example: Simple Buffered CTS in Verilog**

```
// Simple Buffered Clock Tree Example
module clock_tree (
    input wire clk_in,
    output wire clk_ff1,
    output wire clk_ff2
);
    wire clk_buf1;
    wire clk_buf2;

    // Buffer stage 1
    assign clk_buf1 = clk_in;

    // Two branches with buffers
    assign clk_ff1 = clk_buf1; // First branch
    assign clk_ff2 = clk_buf1; // Second branch
endmodule
```

**Note:** In synthesis, actual clock buffers will be inserted automatically based on load balancing and timing constraints.

## ➔ Basic CTS Flow (Conceptual)

**Step 1:** Identify clock sources (e.g., top-level clock ports, PLL outputs).

**Step 2:** Parse netlist to find all clock sinks (flip-flops, latches).

**Step 3:** Insert buffers and balance clock paths.

**Step 4:** Update DEF/netlist with buffer instances.

**Step 5:** Verify skew and latency after CTS.

# **Basic CTS Flow (Conceptual)**

## **Open-Source Equivalent Tools:**

- (RePLAce engine) OpenLane for placement-aware balancing
- FastRoute(Global Routing) ,TritonRoute(Detailed Routing) used for route updates
- Custom TCL scripts for buffer insertion



# ➔ Sample TCL Commands for CTS (Conceptual)

## Conceptual Diagram:

```
# Example TCL pseudo-commands for CTS
set clk_ports [get_ports clk]
set clk_cells [find_clocks -name clk]

insert_buffer -net clk -cell CLKBUF -max_fanout 10
balance_tree -net clk -max_skew 100ps
write_def design_cts.def
```

## Explanation:

- Defines clock net and sinks
- Inserts buffers for fanout control
- Balances tree to limit skew below 100ps

# Clock Tree Quality Metrics

| Metric                 | Description                                                 |
|------------------------|-------------------------------------------------------------|
| <b>Skew</b>            | Difference between earliest and latest clock arrival times. |
| <b>Insertion Delay</b> | Time taken from source to destination.                      |
| <b>Transition Time</b> | Rise/fall time of clock edges.                              |
| <b>Power</b>           | Energy consumed by switching buffers.                       |
| <b>Balance Ratio</b>   | Ratio of clock path delay variation.                        |

**Goal:** Achieve minimum skew ( $< 100$  ps) and balanced insertion delay

## Common CTS Issues

- High Clock Skew: Due to uneven wire lengths.
- Buffer Overload: Excessive fanout → slower transitions.
- Clock Power Consumption: Large number of buffers switching every cycle.
- Crosstalk: Coupling between adjacent clock nets.
- Clock Duty Distortion: Improper buffer rise/fall time ratio.

### Verification Tools:

- Post-CTS STA (Static Timing Analysis) checks clock skew and latency.

# ➔ Summary and Best Practices

## Summary

CTS ensures uniform clock distribution across the chip.

H-tree → ideal for symmetry; Buffered trees → practical for real SoCs.

Balancing load and minimizing skew are key goals.

## Best Practices

Use dedicated clock buffers (CLKBUF cells).

Keep skew  $< 10\%$  of clock period.

Shield critical clock nets from noise.

Verify post-CTS timing and re-run STA after routing.

Keep clock power  $\leq 30\%$  of total chip power.



# Thank You

echiphu

